DELTAPROOF: Finding the Exact Rows That Differ After a Heterogeneous Database Migration

Nithin Narla
Independent Researcher
nithin.nnv@gmail.com

***Abstract*—When a system is re-engineered onto a different database, for example from PostgreSQL to Cassandra, its owners must show that the migrated data matches the original before they cut over. Existing tools compare row counts and aggregates, which miss value errors, or compare every row, which costs a full read of both databases and fails when the two stores order and encode records differently. DELTAPROOF is an open-source command-line tool and Python library that reports the exact keys that are missing, extra or different between two databases. A canonical encoder makes logically equal records hash identically across stores, and each store is summarized by a small sketch built from invertible Bloom lookup tables, whose comparison costs bytes proportional to the number of differences rather than the size of the tables. Sketches can be built next to each store and compared anywhere, and they can be maintained from a change stream while the migration is live. On TPC-H `lineitem` loaded into PostgreSQL and Cassandra, DELTAPROOF found every injected fault with no false positives while exchanging 260 KB for 6 million rows; a checksum of the drivers' raw values flagged every row. Tool: https://github.com/REPLACE-ME/deltaproof.**

***Index Terms*—data migration, reengineering, data validation, set reconciliation, NoSQL**

I. INTRODUCTION

Moving a system's data to a different database is a common reengineering step: a relational store is replaced by a wide-column store to scale writes, or a legacy schema is rebuilt in a new one. The migration is complete only when the target provably holds the same data as the source. In practice teams check row counts or column sums, which pass when a value is rounded, a timestamp is shifted or a lost row is offset by a duplicate, or they read and compare every row, which loads both production databases and grows with table size. Key-range checksum tools such as data-diff [1] and pt-table-checksum [2] reduce transfer but require both stores to scan in the same key order, which hash-partitioned stores such as Cassandra [3] do not provide.

Two problems make heterogeneous verification hard. First, the same logical record has different physical forms: PostgreSQL returns `CHAR(25)` values padded with spaces and Cassandra returns them trimmed; one driver returns a `date`, the other its own date type; decimals, time zones and timestamp precision differ. A checksum over raw values reports nearly every row as different. Second, the useful answer is the list of differing keys, which is usually tiny compared with the tables.

DELTAPROOF addresses both. It treats verification as set reconciliation [5]: each store becomes a set of (key, value digest) elements under a canonical encoding, and the tool computes the symmetric difference of the two sets. It contributes:

- a command-line tool that, given a short table specification and two connection URLs, prints every missing, extra and mismatched key (Section II);
- a canonical record encoder for relational and wide-column stores (Section III);
- a sketch format, built from invertible Bloom lookup tables (IBLTs) [4], that can be produced next to each store, shipped as a file, and compared in bytes proportional to the number of differences;
- an evaluation on PostgreSQL and Cassandra with injected faults (Section IV).

II. USING DELTAPROOF

A. Describing the tables

The user writes a specification that names each table's primary key and its non-key columns with a logical type. The type, not the stores' physical types, decides when two values are equal: `d2` is an exact decimal at scale 2, `c` a fixed-width string whose padding is ignored, `dt` a calendar date, `ts` a UTC timestamp at milliseconds, and a trailing `?` allows nulls. Listing 1 shows the specification used for TPC-H `lineitem`. One specification can list many tables; they are compared together.

Listing 1. Table specification (`lineitem.toml`)

```
[[table]]
name = "lineitem"
key = ["l_orderkey", "l_linenumber"]
columns = """l_partkey:i l_suppkey:i l_quantity:d2
  l_extendedprice:d2 l_discount:d2 l_tax:d2
  l_returnflag:c l_linestatus:c l_shipdate:dt
  l_commitdate:dt l_receiptdate:dt l_shipinstruct:c
  l_shipmode:c l_comment:t"""
```

B. Running a comparison

The simplest use compares two live databases in one command (Listing 2, run with `-limit 1`). When the stores sit in different networks, `deltaproof sketch` runs next to each store and writes a sketch file, and `deltaproof diff` compares the two files anywhere. A file holding every size of sketch is about 20 bytes per row, little less than the rows' digests themselves; with `-max-differences 1000` it holds only the sizes needed for up to 1,000 differences, 232 KB for 600,000 rows. If more differences turn up, `diff` falls back to an exact comparison of the stores named by

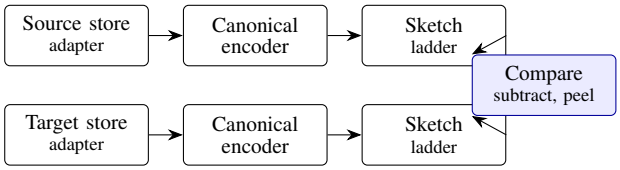


Fig. 1. Pipeline. Each side can run next to its store; only sketches travel.

`--source` and `--target`. The exit status is 0 when the stores match and 1 otherwise, so the command can gate a cutover script or a CI job, and `--json` prints a machine-readable report.

Listing 2. Comparing PostgreSQL with Cassandra

```
$ deltaproof verify --spec lineitem.toml \
    --source postgresql://user:pw@pg:5432/tpch \
    --target cassandra://cass:9042/tpch
120 differing keys
  missing_in_target: 20
    table=lineitem l_orderkey=11655 l_linenumber=3
    ... 19 more (use --limit -1 or --json)
  extra_in_target: 40
    table=lineitem l_orderkey=18277 l_linenumber=2
    ... 39 more (use --limit -1 or --json)
  value_mismatch: 60
    table=lineitem l_orderkey=11683 l_linenumber=5
    ... 59 more (use --limit -1 or --json)
compared 260,496 bytes of sketch in 2 round trips;
rescanned 0 rows
```

C. Verifying while the migration is live

The library exposes the same sketches as objects that accept inserts, updates and deletes. Fed from a change-data-capture stream on the source and from the replicator's writes on the target, they stay equal to sketches rebuilt from scratch, so a migration can be checked every few seconds without rescanning. A watermark and a version check on the target separate writes still in flight from real divergence; this mode, its correctness argument and its evaluation under a TPC-C workload are the subject of a companion paper, and the scripts that run it are in the repository.

III. Architecture

Fig. 1 shows the pipeline. The library and command line are about 1,200 lines of Python with NumPy; baselines and benchmarks are separate.

Adapters read rows through each database's driver (psycopg 3 for PostgreSQL, the DataStax driver for Cassandra) and convert driver values to logical Python values; a new store needs only an adapter that yields rows as dictionaries.

The canonical encoder turns a row into an element: a 128-bit key and a 64-bit digest of the values. Decimals become integers at the declared scale, so `10.5` and `10.50` agree; timestamps become UTC at the declared unit, so a store that keeps microseconds agrees with one that keeps milliseconds; text is Unicode NFC, hashed with BLAKE2b; fixed-width strings drop their padding; floats are compared by their IEEE-754 bits; and nulls carry their own tag, so null, the empty string and zero differ. Composite keys of up to four integer columns are packed with an 8-bit table tag, which lets one sketch cover many tables and lets a reported key be turned back into table and column values.

TABLE I
Same value, different driver forms, one canonical form

Type	PostgreSQL	Cassandra	Canonical
`CHAR(10)`	`'MAIL  '`	`'MAIL'`	trimmed, NFC
date	`date`	`util.Date`	days since 1970
timestamp	µs	naive UTC, ms	UTC, set unit
decimal	column scale	value's scale	scaled integer

Table I lists differences between the two drivers' forms of the same logical value that the encoder absorbs. On `lineitem`, the first two alone made a raw-value checksum flag every row.

Sketches. An IBLT is an array of cells; each element is added to three cells, and each cell keeps a count and XOR sums of keys, digests and a checksum [4]. Subtracting the target's IBLT from the source's cancels every common element, and *peeling* cells that hold a single element recovers the difference. The difference must fit the IBLT, but its size is unknown in advance, so each store keeps a *ladder* of IBLTs at capacities $64 \cdot 8^i$ up to one eighth of the row count, split by key hash into partitions of about 256 differences each, plus a 32-level strata estimator [5]. The comparison reads the estimator first, then the smallest level with 25% headroom over the estimate. A partition that fails to peel is compared row by row, so an undersized estimate costs time, never correctness. The ladder adds about 17 bytes per row.

Engineering choices. Partitioning by key hash rather than key range makes the result independent of each store's scan order. Building every ladder level at once costs little because an update touches three cells per level, and it lets one comparison pick the right size after seeing the estimate. Sketch files of two stores with different row counts remain comparable, because a smaller ladder is a prefix of a larger one. The vectorized NumPy path and the scalar path used while peeling are bit-identical, which the tests check.

IV. Evaluation

We ran PostgreSQL 16.4 and Cassandra 4.1.6 as single nodes in Docker on one Apple M1 Pro laptop (10 cores, 16 GB), with Python 3.13. Faults were injected with SQL and CQL statements, and the ground truth was the exact difference of full scans of both stores.

Does it find the differences? On a 200,000-row synthetic ledger with 120 faults of six classes, and on TPC-H `lineitem` at scale factors 0.1 and 1 with 120 faults (lost writes, decimal precision, shifted dates, corrupted text, lost deletes, phantom rows), DeltaProof reported exactly the injected keys, with no false positives (Table II). A digest of the drivers' raw values, without canonical encoding, flagged 99.9% of ledger rows and every `lineitem` row: the stores format timestamps, padded strings and dates differently. TPC-H data was generated with `dbgen`; these are not TPC-H benchmark results. At SF 0.1 we also ran the `deltaproof`

TABLE II
END-TO-END RUNS, POSTGRESQL VS. CASSANDRA

	Ledger	TPC-H `lineitem`	
		SF 0.1	SF 1
Rows (source)	200,000	600,552	6,001,215
Injected faults	120	120	120
Found / false positives	120 / 0	120 / 0	120 / 0
Sketch bytes compared	260 KB	260 KB	260 KB
Round trips	2	2	2
Raw-value digest flags	99.9%	100%	100%

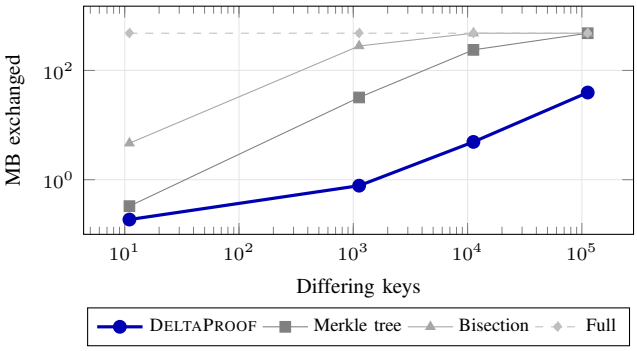


Fig. 2. Bytes exchanged to find the differences among 10^7 rows (log–log).

command itself in all three ways (one command; files with every size; files limited to 1,000 differences), and each printed the same 120 keys.

What does it cost? Comparing sketches took 12 ms for the ledger and 0.16 s for six million rows. The cost of a comparison follows the number of differences (Fig. 2): in a controlled in-process experiment with 10^7 ledger rows and about 1,100 differences, DELTAPROOF exchanged 0.78 MB, while a Merkle tree exchanged 32 MB, range-checksum bisection 280 MB and a full comparison 480 MB, each reporting the same keys.

Building a sketch requires one scan of each store, the same as a full comparison; the savings are in transfer and in later checks, since a sketch maintained from a change stream needs no further scans. In Python, scanning, encoding and sketching `lineitem` took 18 s on PostgreSQL and 28 s on Cassandra at SF 0.1, and 243 s and 495 s at SF 1; this dominates the run time and is the first target for optimization.

Live mode. Under a TPC-C workload replicated from PostgreSQL to Cassandra with injected drops and corruptions, sketches maintained from the change stream equalled full-scan rebuilds in every run, and checks every 10 s reported no false differences; the companion paper reports this experiment in full.

V. RELATED TOOLS

Row counts and aggregates, as offered by several migration services, are cheap but miss value faults. AWS DMS data validation [6] compares rows between a migration task's source and target. Google's Data Validation Tool [7] compares counts, aggregates and row hashes across many databases, though not Cassandra. data-diff [1] and pt-table-checksum [2] checksum key ranges and narrow down mismatches, which needs both sides to scan in key order. Cassandra repairs replicas with Merkle trees [3], within one database and one encoding. DELTAPROOF differs in comparing stores with different encodings and orders, in reporting exact keys from a transfer proportional to the number of differences, and in sketches that can be maintained from a change stream. IBLTs and strata estimators come from network set reconciliation [4], [5]; PinSketch [8] sends fewer bytes at a higher computational cost, and the encoder and pipeline would work with it as well.

VI. LIMITATIONS

Keys must be integer columns (up to four, each below 2^{30}) or UUIDs. Tables without a primary key cannot be compared key by key. The table specification must match the migration's intended mapping; if both sides were encoded wrongly in the same way, a mismatch would be hidden, so specifications need their own tests. Adapters exist for PostgreSQL and Cassandra only. Scans are row-by-row Python and therefore slow on large tables. A run with no differences still exchanges the 187 KB estimator.

VII. CONCLUSION AND AVAILABILITY

DELTAPROOF turns migration verification into a short specification and one command, and returns the exact differing keys from two stores that encode and order data differently. The tool, tests, benchmark scripts, Docker setup and all result files are available under the Apache 2.0 license at https://github.com/REPLACE-ME/deltaproof.

ACKNOWLEDGMENT

The author used Claude (Anthropic) to write the tool's implementation, tests and benchmark scripts and to draft and edit the text of all sections of this paper, including the figure and tables. All results come from running that code. The author reviewed and takes responsibility for all content.

REFERENCES

[1] data-diff, "Technical explanation," project documentation. [Online]. Available: https://data-diff.readthedocs.io/en/latest/technical-explanation.html
[2] Percona, "pt-table-checksum," Percona Toolkit documentation. [Online]. Available: https://docs.percona.com/percona-toolkit/pt-table-checksum.html
[3] A. Lakshman and P. Malik, "Cassandra: A decentralized structured storage system," *ACM SIGOPS Oper. Syst. Rev.*, vol. 44, no. 2, pp. 35–40, 2010.
[4] M. T. Goodrich and M. Mitzenmacher, "Invertible Bloom lookup tables," in *Proc. 49th Annu. Allerton Conf. Commun., Control, Comput.*, 2011, pp. 792–799.
[5] D. Eppstein, M. T. Goodrich, F. Uyeda, and G. Varghese, "What's the difference? Efficient set reconciliation without prior context," in *Proc. ACM SIGCOMM*, 2011, pp. 218–229.
[6] Amazon Web Services, "AWS DMS data validation," AWS Database Migration Service User Guide. [Online]. Available: https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Validating.html
[7] Google Cloud, "Data Validation Tool," GitHub repository. [Online]. Available: https://github.com/GoogleCloudPlatform/professional-services-data-validator
[8] P. Wuille, G. Maxwell, and G. Naumenko, "minisketch: An optimized library for BCH-based set reconciliation," GitHub repository. [Online]. Available: https://github.com/bitcoin-core/minisketch